



武汉大学
Wuhan University

第五章 时序逻辑电路

5.4 时序逻辑电路的设计

5.5 常用时序逻辑电路及应用

武汉大学计算机学院 2024-2025第二学期



目录

Contents

概述

集成触发器

时序逻辑电路的分析

时序逻辑电路的设计

常用时序电路及其应用

5.4 时序逻辑电路的设计方法

同步时序逻辑电路的**设计**是**分析**的**逆过程**, 其任务是根据实际逻辑问题的要求, 设计出能实现给定逻辑功能的电路。

同步时序逻辑电路设计的一般步骤

逻辑抽象:
建立原始
状态图和
状态表



状态
化简



状态
分配



选择
触发
器类
型



确定驱
动方程
组和输
出方程



检查
自启
动能
力



画出
逻辑
电路
图

5.4.1 逻辑抽象

- 确定输入、输出变量及电路的状态数
- 定义输入、输出逻辑状态和电路状态的含义
- 按题意建立原始状态转换图和状态转移真值表

5.4.1 逻辑抽象

【例5.4.1】 设计一个五进制可逆计数器。当输入 $x=0$ 时，进行加1计数；当 $x=1$ 时，进行减1计数

(1) 确定输入、输出变量及电路状态的数量

输入变量：1个

输出变量：1个

电路状态数量：5个

5.4.1 逻辑抽象

【例5.4.1】 设计一个五进制可逆计数器。当输入 $x=0$ 时，进行加1计数；当 $x=1$ 时，进行减1计数

(2) 定义输入、输出变量及电路状态的含义

输入变量： x ——控制电路的功能选择

- $x=0$: 加1计数
- $x=1$: 减1计数

电路状态： S_0-S_4 ——五进制计数器的5种状态

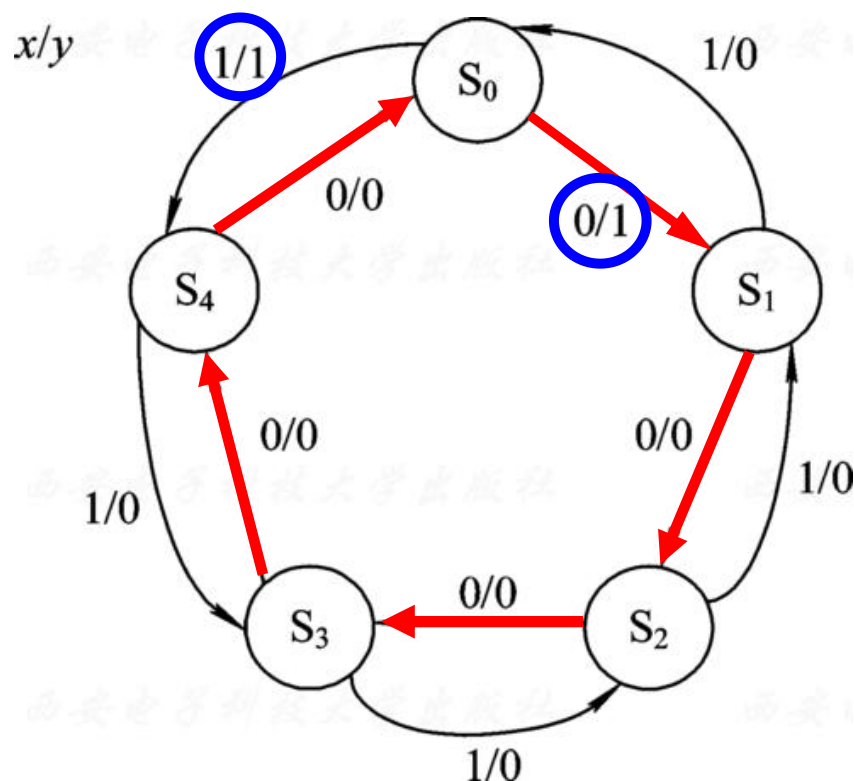
输出变量： y ——指示电路处于初始状态

- $y=1$: 电路处于 S_0 状态
- $y=0$: 电路处于其它状态

5.4.1. 逻辑抽象

【例5.4.1】 设计一个五进制可逆计数器。当输入 $x=0$ 时，进行加1计数；当 $x=1$ 时，进行减1计数

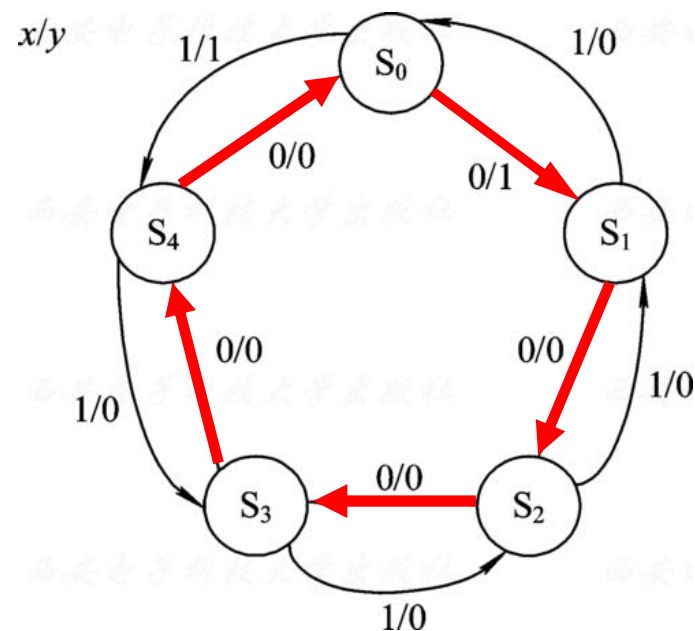
(3) 画出状态转移图



5.4.1 逻辑抽象

【例5.4.1】设计一个五进制可逆计数器。当输入 $x=0$ 时，进行加1计数；当 $x=1$ 时，进行减1计数

(4) 列出状态转移真值表



次态/输出 现态		x	
		0	1
S_0		$S_1 / 1$	$S_4 / 1$
S_1		$S_2 / 0$	$S_0 / 0$
S_2		$S_3 / 0$	$S_1 / 0$
S_3		$S_4 / 0$	$S_2 / 0$
S_4		$S_0 / 0$	$S_3 / 0$

5.4.2 状态化简和分配

【例5.4.2】 设计一个串行数据检测器， 在连续输入四个或四个以上的1时输出1， 否则输出0。

解 设计一个“1111”序列检测器。

输入 X ， 表示串行输入数据。

输出 Z ， 表示检测结果。

电路状态定义：

S_0 ——未输入1， Z ——输出0；

S_1 ——输入一个1， Z ——输出0；

S_2 ——连续输入两个1， Z ——输出0；

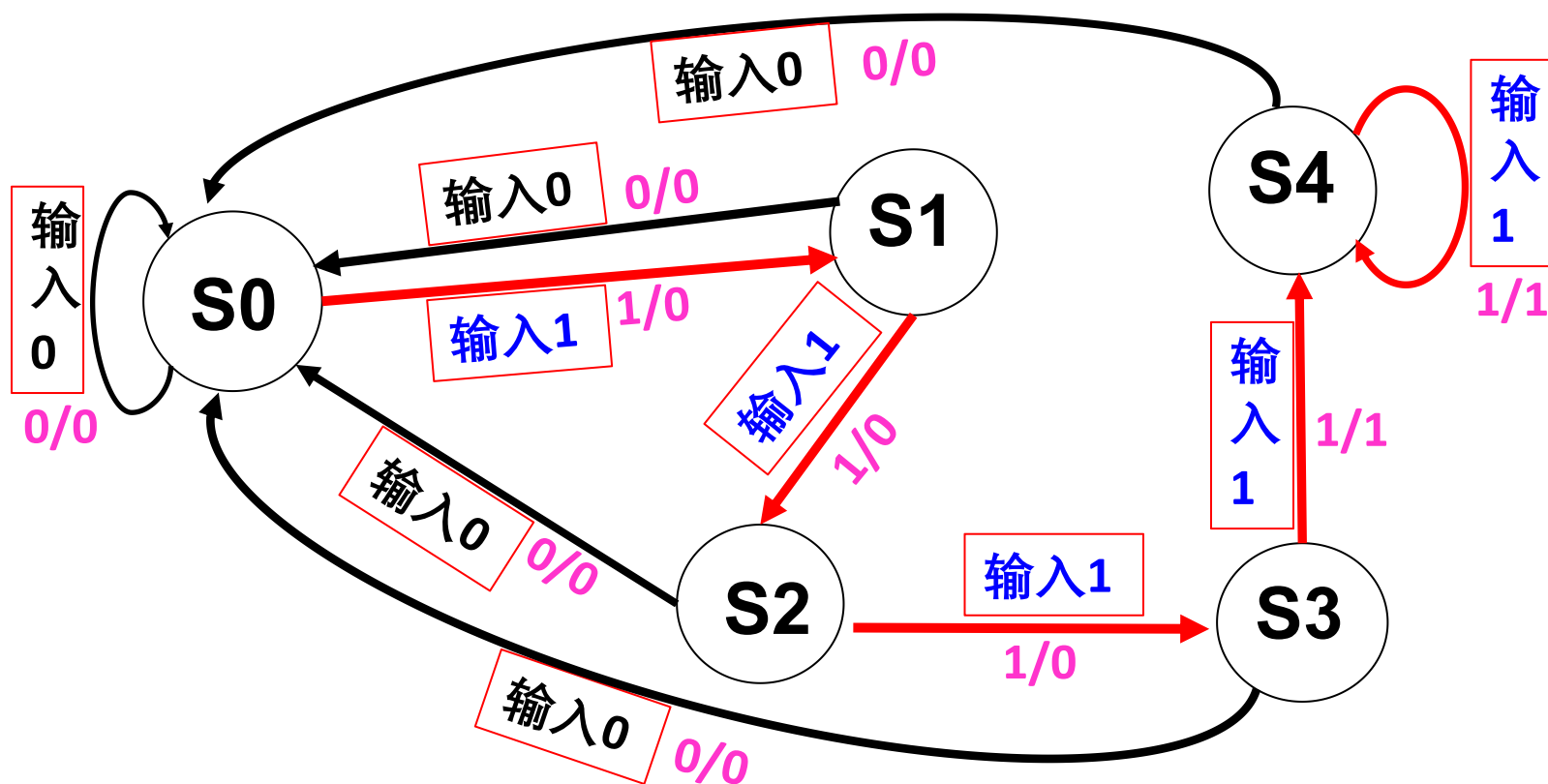
S_3 ——连续输入三个1， Z ——输出0；

S_4 ——连续输入四个或四个以上1， Z ——输出1。

5.4.1 逻辑抽象

【例5.4.2】设计一个串行数据检测器，在连续输入四个或四个以上的1时输出1，否则输出0。

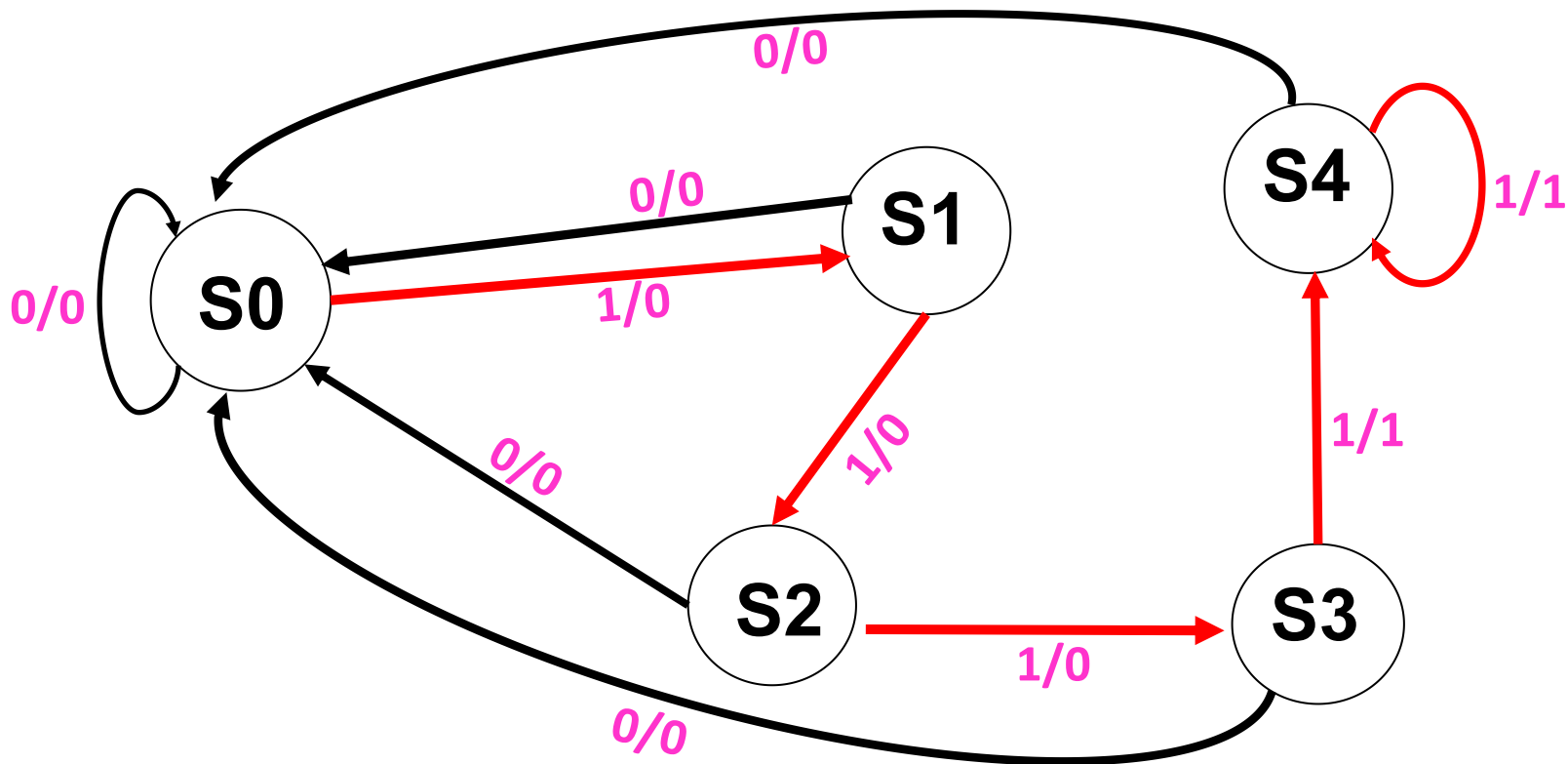
(2) 画出状态转移图



5.4.1 逻辑抽象

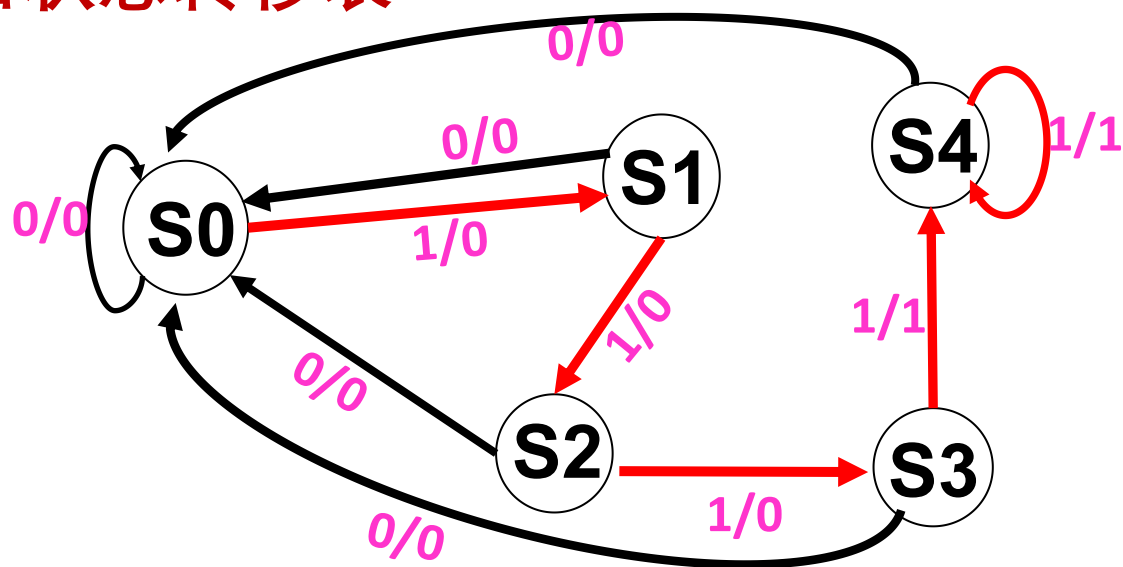
【例5.4.2】设计一个串行数据检测器，在连续输入四个或四个以上的1时输出1，否则输出0。

(2) 画出状态转移图



5.4.1 逻辑抽象

(3) 列出状态转移表



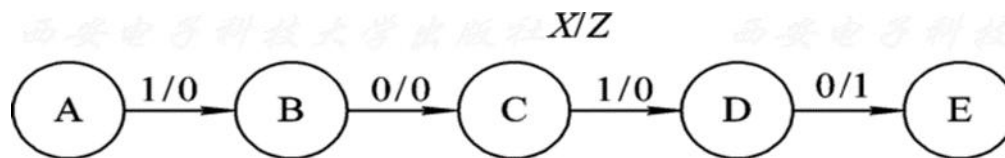
$S^{n+1} / Z \backslash S^n$	S_0	S_1	S_2	S_3	S_4
X					
0	$S_0 / 0$	$S_0 / 0$	$S_0 / 0$	$S_0 / 0$	$S_0 / 0$
1	$S_1 / 0$	$S_2 / 0$	$S_3 / 0$	$S_4 / 1$	$S_4 / 1$

5.4.2 状态化简和分配

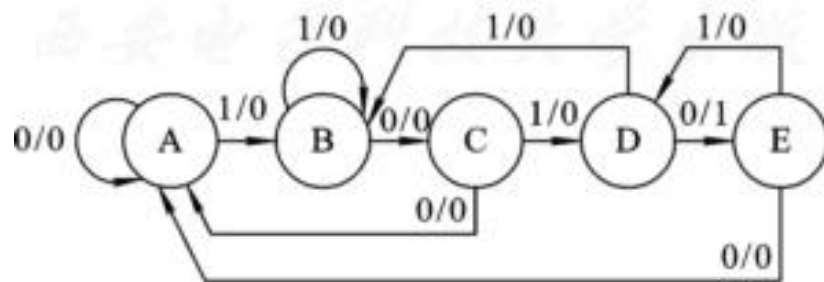
【例5.4.3】 用树干分支法画出重叠型和非重叠型“1010”序列检测器的原始状态图。

重叠型序列检测器：前、后序列是可以重叠的，如重叠型“1010”序列在已经收到“1010”后，只要再收到“10”就表示检测到了下一个“1010”序列，因为是用上一个序列的后一个“10”作为下一序列的前两位“10”的。

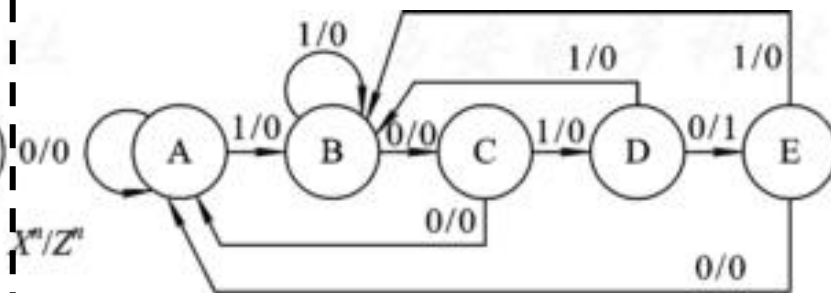
非重叠型序列检测器：因已经检测到了“1010”，所以后面的两位“10”不可再用，应从下一位开始重新检测“1010”。



5.4.2 状态化简和分配



(a) 重叠型



(b) 非重叠型

5.4.2 状态化简和分配

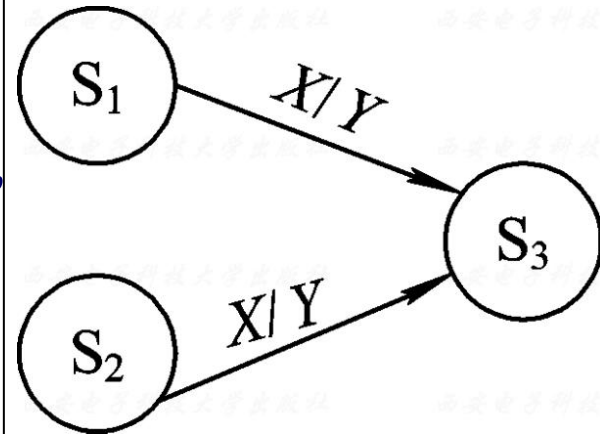
合并等效状态、消去多余状态的过程，利用状态与状态间的等效关系进行状态简化

状态等效：

在完全确定的状态表中，若对所有的可能的输入序列，分别从某两个状态（假设为S1与S2）出发，所得到的输出序列都相同，且状态转移效果相同，则说这两个状态（S1与S2）等效。记做：

(S1, S2)

此时S1与S2可以合并为一个状态



5.4.2 状态化简和分配

等效的传递性:

若有状态S1和S2等效, 状态S2和S3等效, 则状态S1和S3也等效, 记为:

$$(S1, S2), (S2, S3) \rightarrow (S1, S3)$$

等效类:

所含状态都可以互相构成等效对的等效状态集合, 即:

$$(S1, S2, S3) \rightarrow (S1, S2) (S2, S3) (S1, S3)$$

$$(S1, S2) (S2, S3) (S1, S3) \rightarrow (S1, S2, S3)$$

最大等效类:

一个原始状态表中, 不能被其它等效类所包含的等效类

5.4.2 状态化简和分配

等效对的判断标准:

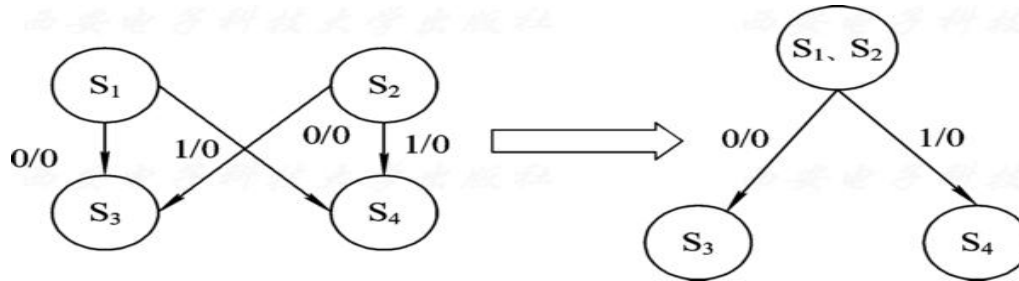
条件1: 在输入相同的条件下, 其输出完全相同。

条件2: 在条件1的基础上, 每个次态满足下列条件之一。

等效对:

次态相同; 次态交错; 次态维持; 后继状态等效; 次态循环。

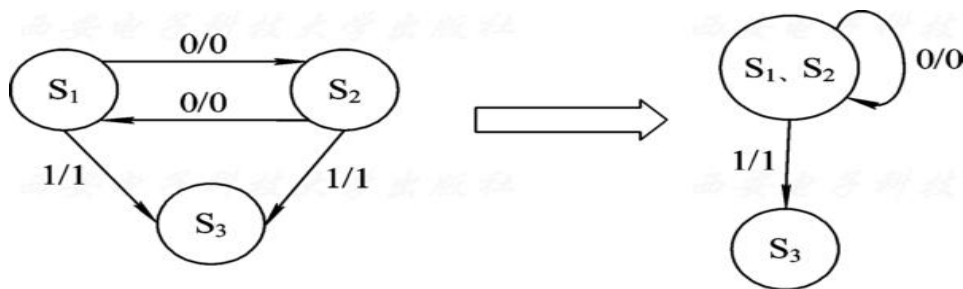
次态相同: 状态 S_1 , S_2 在输入相同的情况下, 输出均相同, 且状态转移效果均相同。



次态相同示意图

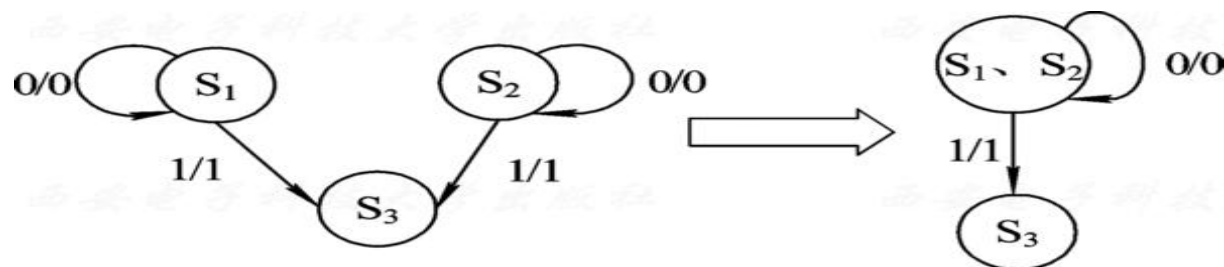
5.4.2 状态化简和分配

次态交错： 次态交错是指状态 S_i 的次态是 S_j ， 状态 S_j 的次态是 S_i 。



次态交错示意图

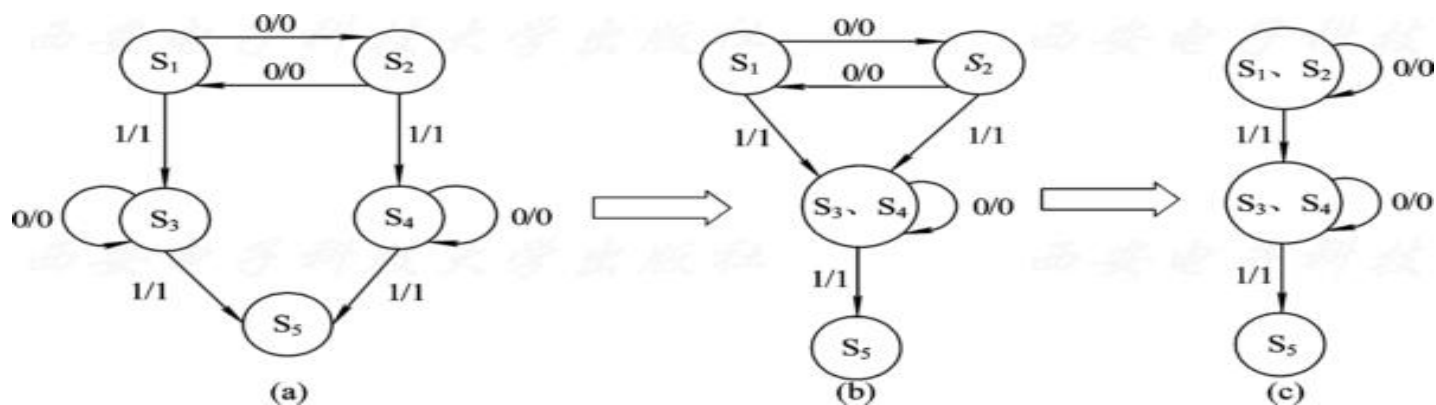
次态维持： 次态维持是指在某些输入相同的情况下， S_i 、 S_j 的次态维持其本身不变， 则 S_i 、 S_j 是等价态。



次态维持示意图

5.4.2 状态化简和分配

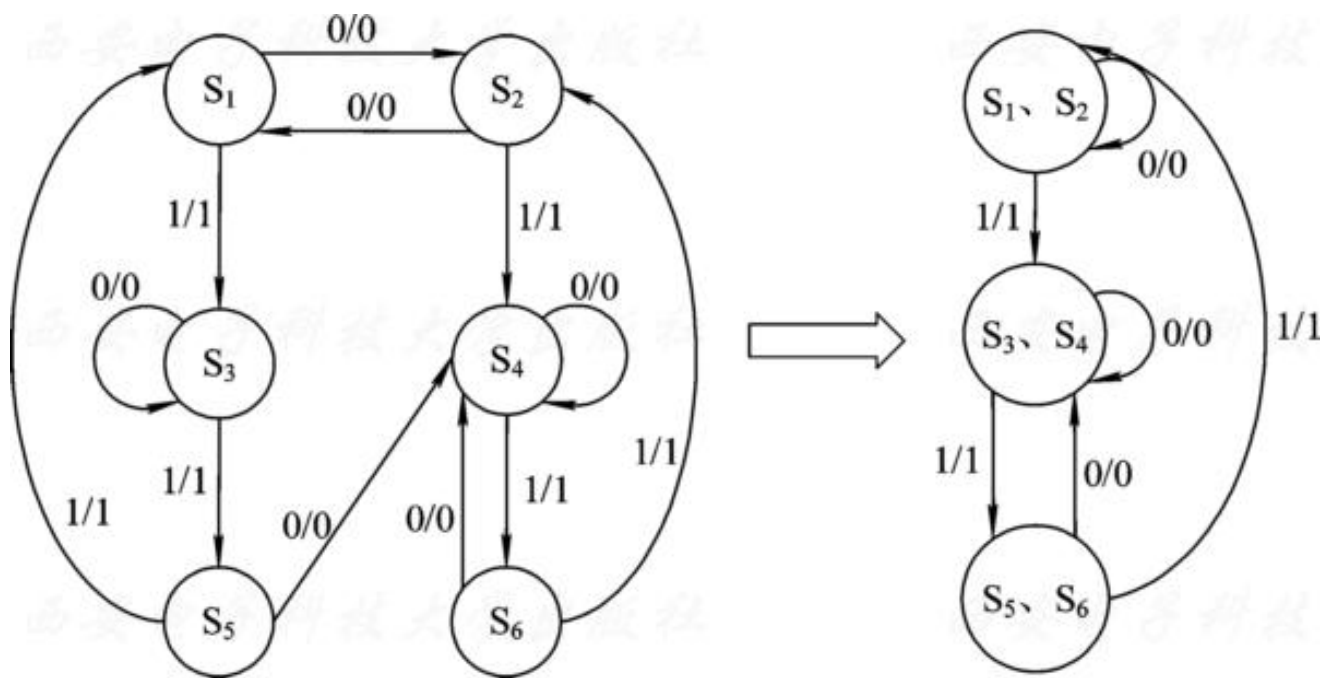
后继状态等效： 后继状态等效是指若两个状态的后继态是等效态，则这两个状态也是等效态。



后继状态等效示意图

5.4.2 状态化简和分配

次态循环：次态之间相互循环，是指状态 S_1 和 S_2 等价的前提条件是状态 S_3 和 S_4 等价，而 S_3 和 S_4 等价的前提条件又是状态 S_1 和 S_2 等价，此时， S_1 和 S_2 等价， S_3 和 S_4 也等价。

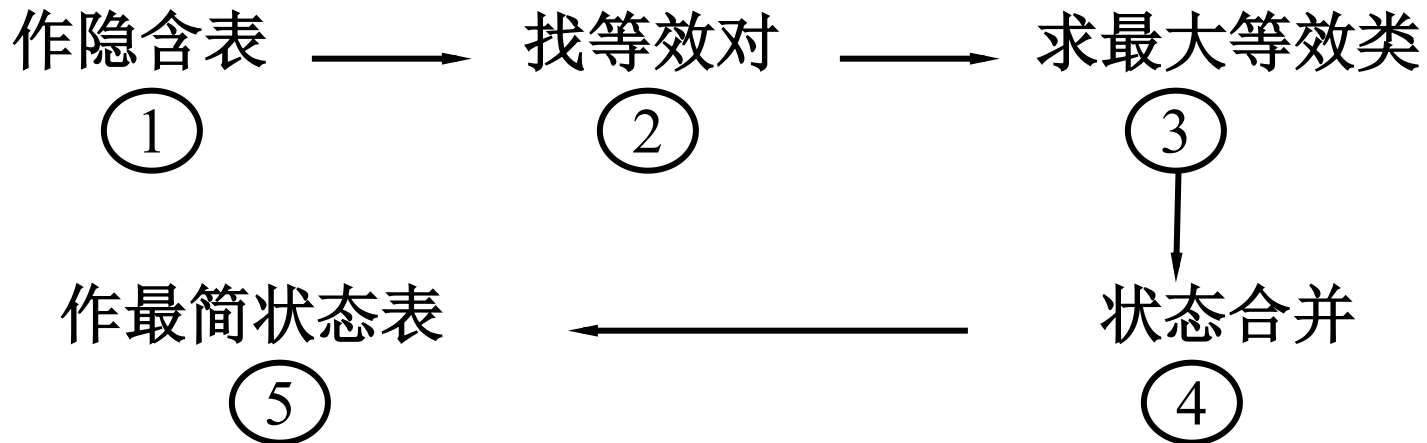


次态循环示意图

5.4.2 状态化简和分配

3) 用隐含表法进行状态化简：对于简单的等价态关系，可以直接通过观察进行判断；对于比较复杂的对等关系，可以采用隐含表法进行状态化简。

隐含表化简法的一般步骤



5.4.2 状态化简和分配

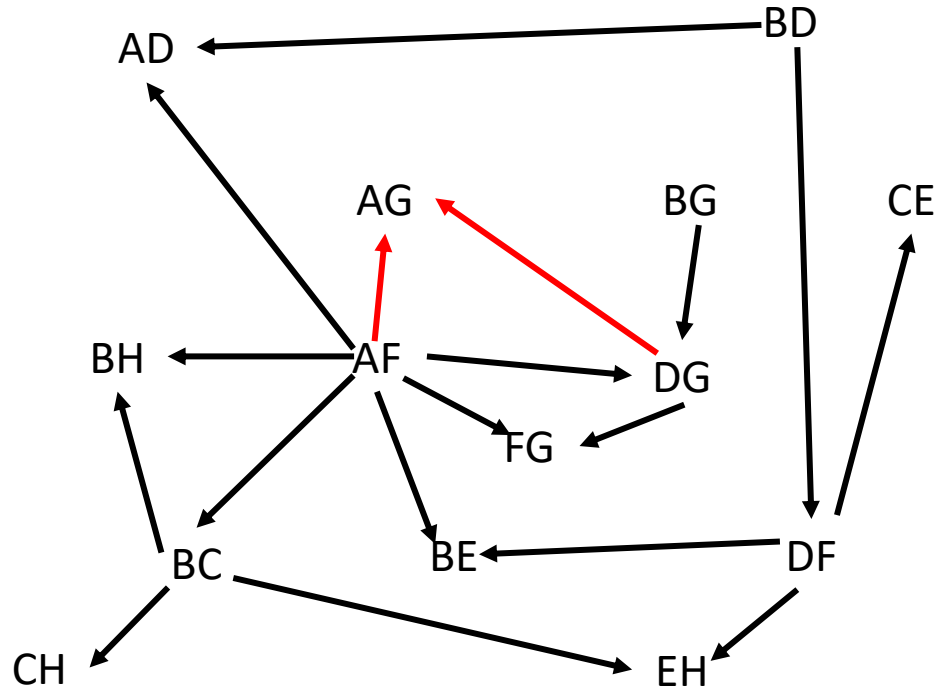
隐含表

次态/输出 现态 \ 输入	00	01	10	11
A	D/ <u>0</u>	D/ <u>0</u>	F/ <u>0</u>	A/ <u>0</u>
B	C/1	D/0	E/1	F/0
C	C/1	D/0	E/1	A/0
D	D/ <u>0</u>	B/ <u>0</u>	A/ <u>0</u>	F/ <u>0</u>
E	C/1	F/0	E/1	A/0
F	D/0	D/0	A/0	F/0
G	G/0	G/0	A/0	A/0
H	B/1	D/0	E/1	A/0

B	×						
C	×	AF					
D	AF BD	×	×				
E	×	AF DF	DE	×			
F	√	×	×	BD	×		
G	AF DG	×	×	AF BG	×	AF DG	
H	×	AF BC	BC	×	BC DE	×	×
	A	B	C	D	E	F	G

- ①相同输入时输出不同：在对应的小方格中填入“×”。
- ②对应所有输入其输出均相同，转移的状态不同：填入待定状态对。
- ③对应所有输入其输出均相同，转移的状态等效：填入“√”。例状态AF
- ④重复扫描，通过已确定状态的状态对，判别待定状态对的状态状况，直至没有待定状态。

5.4.2 状态化简和分配



根据前表的示意，逐列/行一个个画出其待定的状态对：

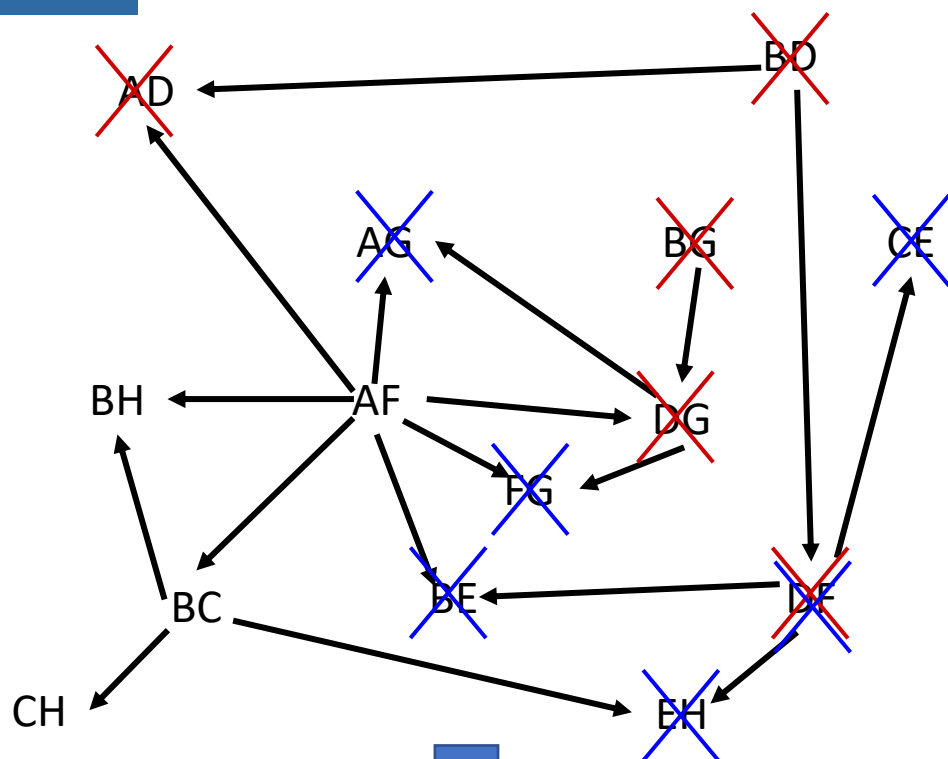
如：第一列，AB、AC、AE已打叉，略过。

AG，取决于AF和DG，则构成待定状态对（标红的箭头）

箭头指向的是待判断的等效对

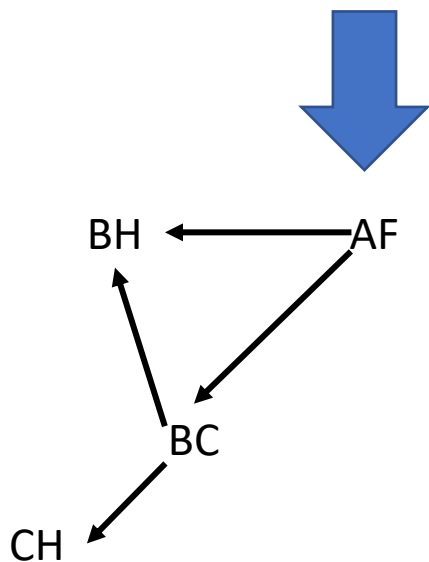
以此类推，把表转换成上述“待定等效对”

5.4.2 状态化简和分配



从表中确定的不等效对，在图中进行删除。（红色 X）

根据红色×确定的不等效对，进一步通过其箭头指向，消去关联的状态对（蓝色 X）



5.4.2 状态化简和分配

⑤找出等效对：(A, F)、(B, C)、(B, H)和(C, H)
列出最大等效类：

(A, F)、(B, C, H)、(D)、(E)和(G)

对原始各状态重新命名：

(A, F)→A', (B, C, H)→B', (D)→C', (E)→D', (G)→E'

⑥列出化简后的最小化状态表,

次态 现态	输入			
	00	01	10	11
A'	C'/0	C'/0	A'/0	A'/0
B'	B'/1	C'/0	D'/1	A'/0
C'	C'/0	B'/0	A'/0	A'/0
D'	B'/1	A'/0	D'/1	A'/0
E'	E'/0	E'/0	A'/0	A'/0

5.4.2 状态化简和分配

- 状态分配就是给最小化状态表中的每个状态分配一个**二进制代码**
- 电路的状态是触发器状态的组合。如果电路有 M 个状态， $2^{n-1} < M \leq 2^n$ ，则可用 n 个触发器来实现

在 $M < 2^n$ 个状态中选取 M 个状态的组合方案共有：
$$C_{2^n}^M = \frac{2^n!}{M!(2^n - M)!}$$

而 M 个状态的排列又有 $M!$ 种，因而状态编码的方案共有：

$$M!C_{2^n}^M = \frac{2^n!}{(2^n - M)!}$$

5.4.2 状态化简和分配

分配状态方法：

(1) 找出状态之间的相邻关系。可按照如下三个规则进行。

规则 1：在相同输入条件下，次态相同，现态相邻。

规则 2：在相邻输入条件下，同一现态，次态相邻。

规则 3：输出完全相同，现态相邻

这三条规则在用于实际分配时可能会产生矛盾，此时应按照规则1、规则2、规则3的顺序进行分配。

(2) 给各状态分配二进制编码：

① 找出状态表中出现最多的次态 S_{j+1} 所对应的现态 S_i ，并令 S_i 的二进制编码全为 0。

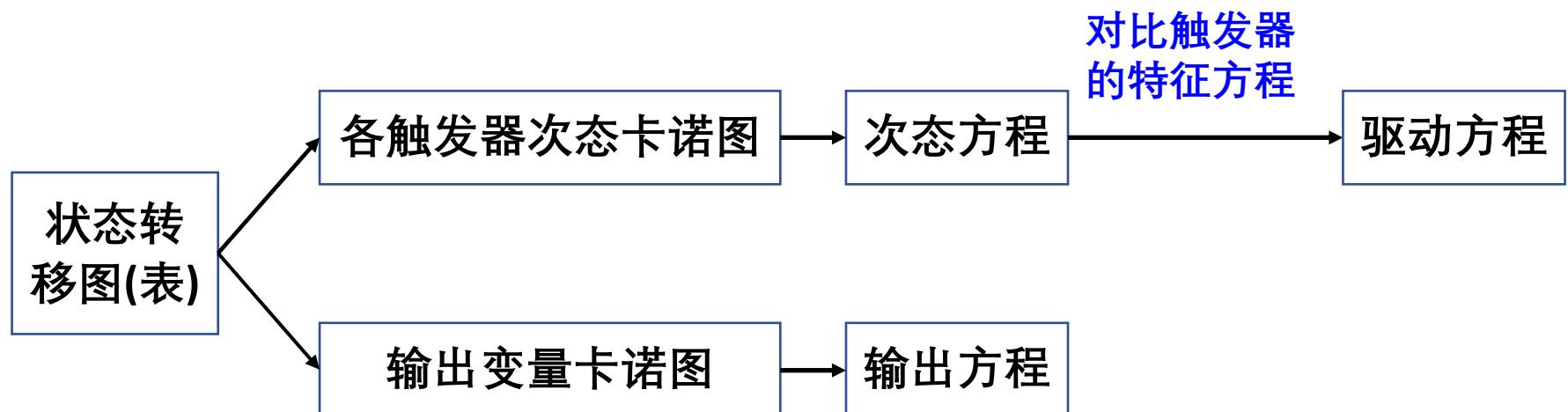
② 按(1)已确定出的状态相邻关系给其他状态分配二进制编码。

5.4.3 确定驱动方程组和输出方程

在确定了触发器类型的条件下，再确定驱动方程组。

最常用的触发器是D触发器，其次是J-K触发器和T触发器。

由状态转移表得到驱动方程组和输出方程的过程：



5.4.3 确定驱动方程组和输出方程

例3 分别用**D触发器**、**J-K触发器**和**T触发器**实现下表所示的逻辑功能，试求对应的驱动方程和输出方程。

现态 $Q_2^n Q_1^n$	$x=0$	$x=1$
	$Q_2^{n+1} Q_1^{n+1} / Z$	$Q_2^{n+1} Q_1^{n+1} / Z$
00	10/0	11/0
01	10/0	00/0
11	00/0	01/1
10	01/0	11/0

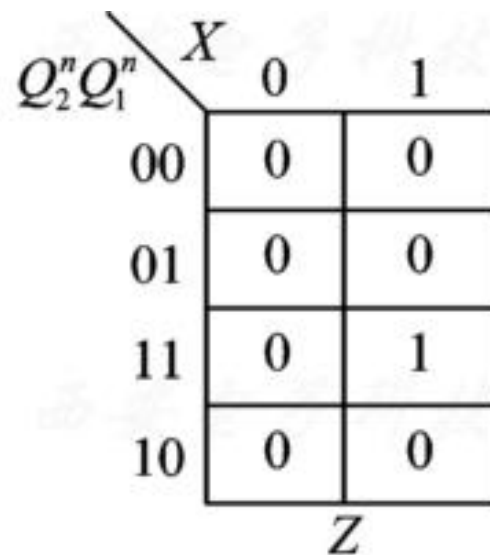
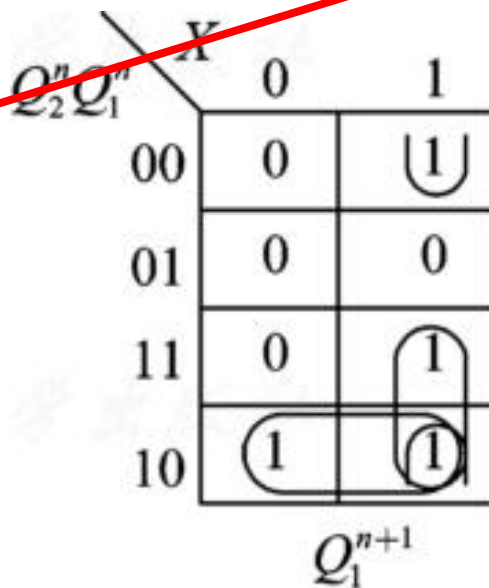
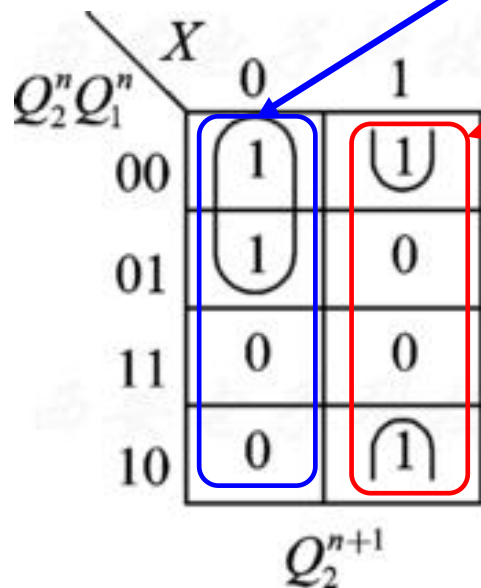
5.4.3 确定驱动方程组和输出方程

(1) 绘制触发器次态卡诺图和输出卡诺图

次态和输出信号都是现态和输入信号的函数。

现态 $Q_2^n Q_1^n$	$x=0$	$x=1$
	$Q_2^{n+1} Q_1^{n+1} / Z$	$Q_2^{n+1} Q_1^{n+1} / Z$
00	10/0	11/0
01	10/0	00/0
11	00/0	01/1
10	01/0	11/0

按循
环码
顺序
排列



5.4.3 确定驱动方程组和输出方程

(2) 求出次态方程和输出方程

$Q_2^n Q_1^n$		X	
		0	1
00	1	1	1
01	1	0	0
11	0	0	0
10	0	1	1

Q_2^{n+1}

$Q_2^n Q_1^n$		X	
		0	1
00	0	0	1
01	0	0	0
11	0	1	1
10	1	1	1

Q_1^{n+1}

$Q_2^n Q_1^n$		X	
		0	1
00	0	0	0
01	0	0	0
11	0	1	1
10	0	0	0

Z

$$\begin{cases} Q_2^{n+1} = \overline{X} \overline{Q_2^n} + X \overline{Q_1^n} \\ Q_1^{n+1} = X Q_2^n + X \overline{Q_1^n} + Q_2^n \overline{Q_1^n} \end{cases}$$

$$Z = X Q_2^n Q_1^n$$

4. 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用D触发器

$$Q^{n+1} = D$$

$$\begin{cases} Q_2^{n+1} = \overline{X} \overline{Q_2^n} + X \overline{Q_1^n} \\ Q_1^{n+1} = X Q_2^n + X \overline{Q_1^n} + Q_2^n \overline{Q_1^n} \end{cases}$$

对比，得到

$$\begin{cases} D_2 = \overline{X} \overline{Q_2^n} + X \overline{Q_1^n} \\ D_1 = X Q_2^n + X \overline{Q_1^n} + Q_2^n \overline{Q_1^n} \end{cases}$$

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用J-K触发器

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

$$\begin{aligned}
 Q_2^{n+1} &= \overline{X}\overline{Q_2^n} + X\overline{Q_1^n} \\
 &= \overline{X}\overline{Q_2^n} + X\overline{Q_1^n}(\overline{Q_2^n} + Q_2^n) \\
 &= (\overline{X} + X\overline{Q_1^n})\overline{Q_2^n} + X\overline{Q_1^n}Q_2^n \\
 &= (\overline{X} + \overline{Q_1^n})\overline{Q_2^n} + X\overline{Q_1^n}Q_2^n
 \end{aligned}$$

对比



$$\begin{cases}
 J_2 = \overline{X} + \overline{Q_1^n}, \\
 K_2 = \overline{X\overline{Q_1^n}} = \overline{X} + Q_1^n
 \end{cases}$$

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用J-K触发器

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

$$\begin{aligned} Q_1^{n+1} &= XQ_2^n + X\overline{Q_1^n} + Q_2^n\overline{Q_1^n} \\ &= XQ_2^n(Q_1^n + \overline{Q_1^n}) + X\overline{Q_1^n} + Q_2^n\overline{Q_1^n} \\ &= (X + Q_2^n + XQ_2^n)\overline{Q_1^n} + XQ_2^nQ_1^n \\ &= (X + Q_2^n)\overline{Q_1^n} + XQ_2^nQ_1^n \end{aligned}$$

对比

$$\begin{cases} J_1 = X + Q_2^n, \\ K_1 = \overline{XQ_2^n} = \overline{X} + \overline{Q_2^n} \end{cases}$$

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用T触发器 $Q^{n+1} = T\overline{Q}^n + \overline{T}Q^n$

T触发器激励表

$Q^n \rightarrow Q^{n+1}$		T
0	0	0
0	1	1
1	0	1
1	1	0

分析：

T触发器要发生
状态翻转，必须
满足输入端信号

T=1

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用T触发器

假设两个T触发器的输入端信号分别为T1和T2，
由状态转移表画T1和T2的卡诺图

现态 $Q_2^n Q_1^n$	$x=0$	$x=1$
	$Q_2^{n+1} Q_1^{n+1} / Z$	$Q_2^{n+1} Q_1^{n+1} / Z$
00	10/0	11/0
01	10/0	00/0
11	00/0	01/1
10	01/0	11/0

Diagram illustrating state transitions for two T flip-flops. Red arrows represent transitions for T1 and blue arrows represent transitions for T2. A red arrow from 10 to 01 is labeled '0'.

$Q_2^n Q_1^n \backslash X$	0	1
00	1	1
01	1	0
11	1	1
10	1	0

T2

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用T触发器

假设两个T触发器的输入端信号分别为T1和T2，由状态转移表画T1和T2的卡诺图

现态 $Q_2^n Q_1^n$	$x=0$	$x=1$
	$Q_2^{n+1} Q_1^{n+1} / Z$	$Q_2^{n+1} Q_1^{n+1} / Z$
00	10/0	11/0
01	10/0	00/0
11	00/0	01/1
10	01/0	11/0

Diagram illustrating state transitions for two T flip-flops. Red arrows indicate transitions for T_1 and blue arrows indicate transitions for T_2 . The output Z is shown in the second column of the table.

$Q_2^n Q_1^n \backslash X$	0	1
00	0	1
01	1	1
11	1	0
10	1	1

T1

5.4.3 确定驱动方程组和输出方程

(3) 求驱动方程

◆ 若选用T触发器

对T1和T2的卡诺图进行化简，得到T1和T2的逻辑表达式

$Q_2^n Q_1^n \backslash X$	0	1
00	0	1
01	1	1
11	1	0
10	1	1

T1

$Q_2^n Q_1^n \backslash X$	0	1
00	1	1
01	1	0
11	1	1
10	1	0

T2

$$\begin{cases} T_2 = \overline{X} + \overline{Q_2^n} \overline{Q_1^n} + Q_2^n Q_1^n \\ T_1 = \overline{X} Q_1^n + X \overline{Q_2^n} + Q_2^n \overline{Q_1^n} \end{cases}$$

5.4.4 检查自启动功能

当所设计的电路中，触发器能表示的状态数大于有效状态数时，需要对所设计的电路进行实际工作状态讨论。

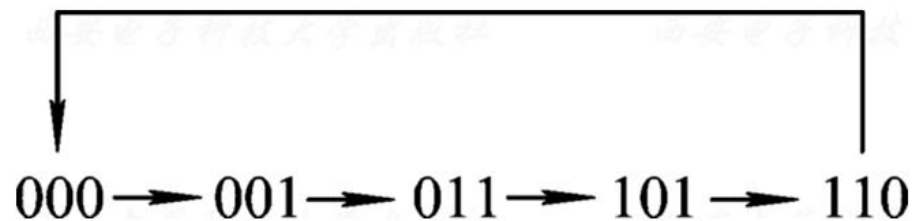
讨论两个问题：

- (1) 电路万一进入无用状态，能否在时钟脉冲作用下进入有效状态，即是否具有自恢复功能和自启动功能。
- (2) 电路万一处在无用状态，是否会产生错误信号。

一但出现以上两个问题都必须修改电路设计，否则将影响电路工作的可靠性和正确性。

5.4 时序逻辑电路的设计方法

例 用J-K触发器设计一个五进制**同步**计数器，要求状态转移关系如右图所示。



解：(1) **逻辑抽象，建立原始状态图和状态表。**

本例属于给定状态的时序电路设计问题，不存在逻辑抽象的问题，可以直接建立**状态转移真值表**。

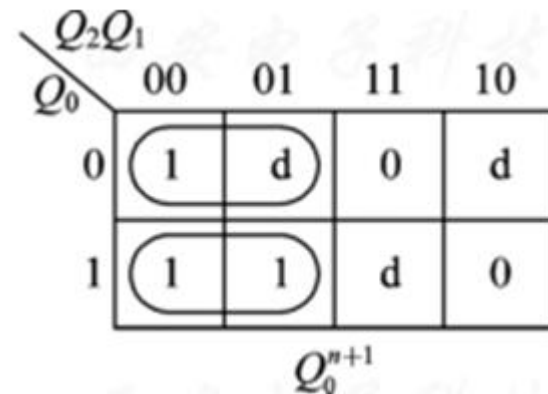
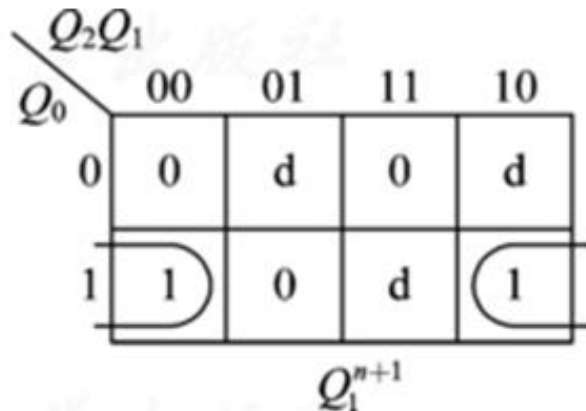
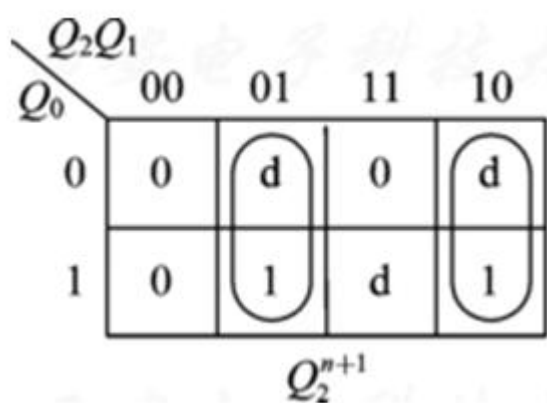
Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	×	×	×
0	1	1	1	0	1
1	0	0	×	×	×
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	×	×	×

**多余
状态**

5.4 时序逻辑电路的设计方法

解：（2）**选择触发器，确定驱动方程和输出方程。**
根据状态转移真值表，画出各个次态的卡诺图。

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	×	×	×
0	1	1	1	0	1
1	0	0	×	×	×
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	×	×	×



5.4 时序逻辑电路的设计方法

解： (2) **选择触发器，确定驱动方程和输出方程。**

Q_2Q_1	00	01	11	10
Q_0				
0	0	d	0	d
1	0	1	d	1

Q_2^{n+1}

Q_2Q_1	00	01	11	10
Q_0				
0	0	d	0	d
1	1	0	d	1

Q_1^{n+1}

Q_2Q_1	00	01	11	10
Q_0				
0	1	d	0	d
1	1	1	d	0

Q_0^{n+1}

根据卡诺图，写出个触发器次态方程和电路输出方程：

$$\begin{cases} Q_2^{n+1} = Q_1^n \overline{Q_2^n} + \overline{Q_1^n} Q_2^n \\ Q_1^{n+1} = Q_0^n \overline{Q_1^n} \\ Q_0^{n+1} = \overline{Q_2^n} = \overline{Q_2^n} \overline{Q_0^n} + \overline{Q_2^n} Q_0^n \end{cases}$$

若选用J-K触发器，则由次态卡诺图求出其状态方程和驱动方程：

$$\begin{cases} J_2 = Q_1^n, & K_2 = Q_1^n \\ J_1 = Q_0^n, & K_1 = 1 \\ J_0 = \overline{Q_2^n}, & K_0 = Q_2^n \end{cases}$$

5.4 时序逻辑电路的设计方法

解：(3) 自启动检查

$$\begin{cases} Q_2^{n+1} = Q_1^n \overline{Q_2^n} + \overline{Q_1^n} Q_2^n \\ Q_1^{n+1} = Q_0^n \overline{Q_1^n} \\ Q_0^{n+1} = \overline{Q_2^n} = \overline{Q_2^n} \overline{Q_0^n} + \overline{Q_2^n} Q_0^n \end{cases}$$

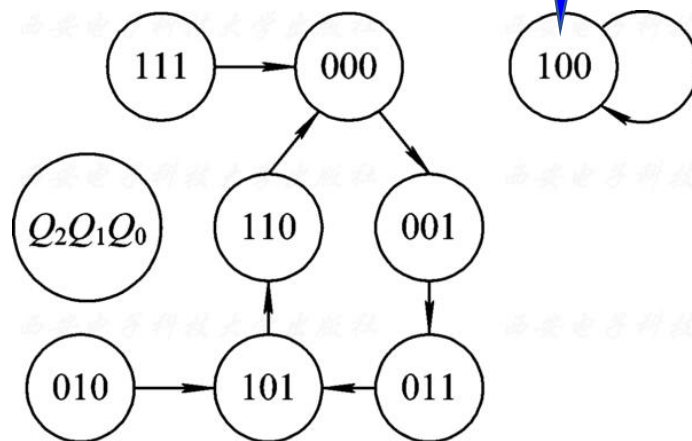
根据以上次态方程，检查多余状态的转移情况。

状态转移表

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	1	0	1
0	1	1	1	0	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	0	0	0

不能自启

状态转移图

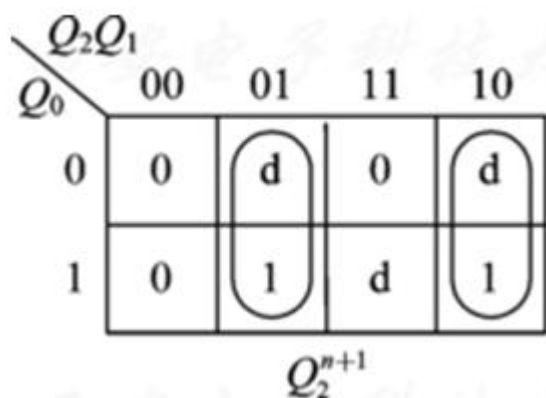


5.4 时序逻辑电路的设计方法

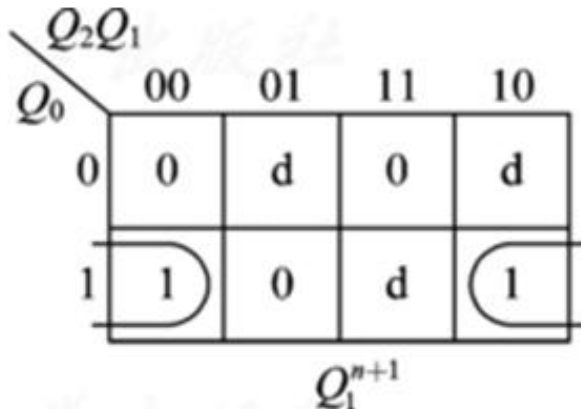
解：(3) **自启动检查**

修改电路设计，避免不能自启动的问题。

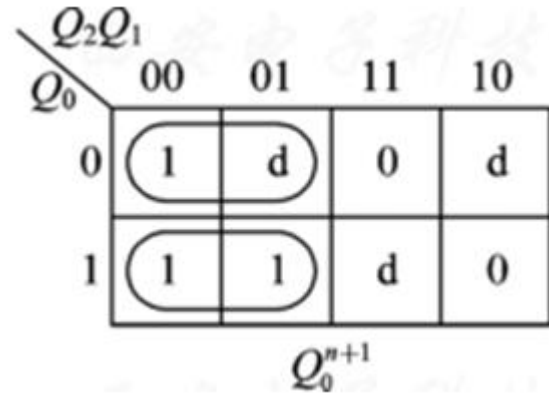
方法：改变卡诺图的圈法



(a)

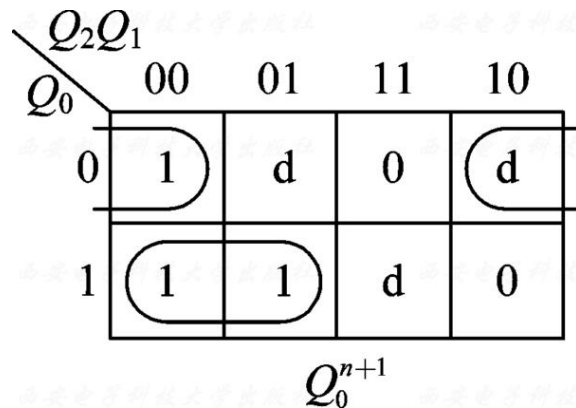


(b)



(c)

如果希望能尽量使用任意项，那么只能对图(a)和图(c)的圈法作修改。现选择对图(c)的圈法作修改，即仅改变 Q_0 的转移，



改进后的 Q_0 卡诺图

5.4 时序逻辑电路的设计方法

解： (3) 自启动检查

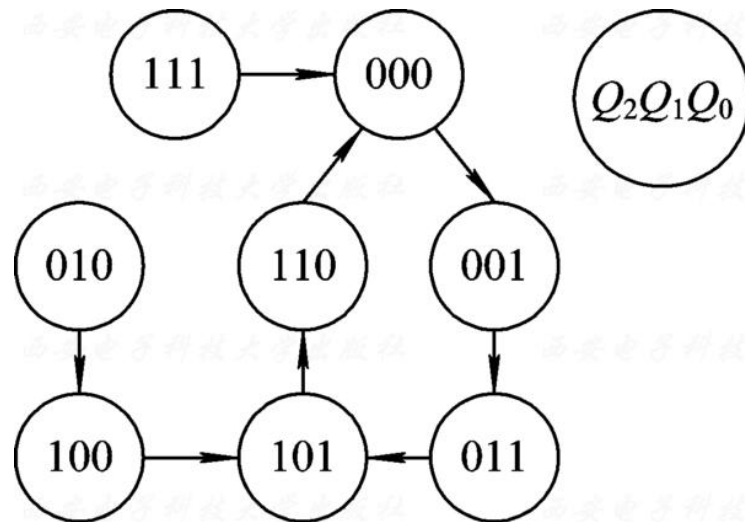
Q_2Q_1 Q_0	00	01	11	10
0	1	d	0	d
1	1	1	d	0

Q_0^{n+1}

改进后的 Q_0 卡诺图

$$\begin{cases} Q_0^{n+1} = \overline{Q_1^n} \overline{Q_0^n} + \overline{Q_2^n} Q_0^n \\ J_0 = \overline{Q_1^n} \\ K_0 = Q_2^n \end{cases}$$

$$\begin{cases} Q_2^{n+1} = Q_1^n \overline{Q_2^n} + \overline{Q_1^n} Q_2^n, & J_2 = Q_1^n, & K_2 = Q_1^n \\ Q_1^{n+1} = Q_0^n \overline{Q_1^n}, & J_1 = Q_0^n, & K_1 = 1 \\ Q_0^{n+1} = \overline{Q_1^n} \overline{Q_0^n} + \overline{Q_2^n} Q_0^n, & J_0 = \overline{Q_1^n}, & K_0 = Q_2^n \end{cases}$$

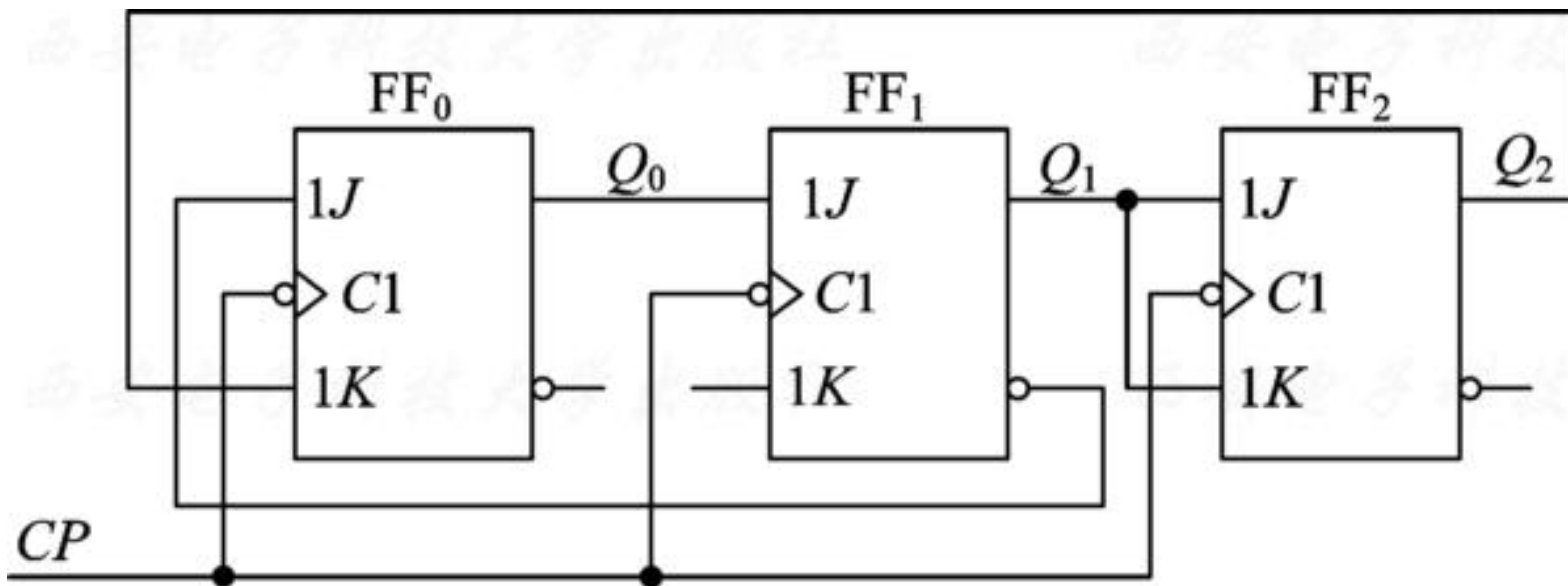


能够自启动的状态转移图

5.4 时序逻辑电路的设计方法

解： (4) 画出电路图

$$\begin{cases} Q_2^{n+1} = Q_1^n \overline{Q_2^n} + \overline{Q_1^n} Q_2^n, & J_2 = Q_1^n, & K_2 = Q_1^n \\ Q_1^{n+1} = Q_0^n \overline{Q_1^n}, & J_1 = Q_0^n, & K_1 = 1 \\ Q_0^{n+1} = \overline{Q_1^n} \overline{Q_0^n} + \overline{Q_2^n} Q_0^n, & J_0 = \overline{Q_1^n}, & K_0 = Q_2^n \end{cases}$$



目录

Contents

概述

集成触发器

时序逻辑电路的分析

时序逻辑电路的设计

常用时序电路及其应用

5.5.1 计数器

计数：累计输入**脉冲**的个数

构成：

- 1个触发器有2个状态，可计2个数(1位二进制数)
- n 个触发器有 2^n 个状态，可计 2^n 个数(n 位二进制数)

应用：分频、定时、产生节拍脉冲和脉冲序列、进行数字运算等

5.5.1 计数器

分类

按数制分：

二进制计数器
十进制计数器
 N 进制 (任意进制) 计数器

按计数
方式分：

加法计数器
减法计数器
可逆计数 (Up-Down Counter)

按时钟
控制分：

同步计数器 (Synchronous ~)
异步计数器 (Asynchronous ~)

同步计数器的特点：

在同步计数器内部，各个触发器都受**同一**时钟脉冲——输入计数脉冲的控制，因此，它们**状态的更新几乎是同时的**，故被称为“同步计数器”

1. 常用集成计数器

(1) 4位二进制加法计数器:

74LS161和74LS163

(2) 4位十进制加法计数器:

74LS160和74LS162

(3) 4位十进制加/减法计数器:

74LS190和74LS192

(4) 4位同步二进制加/减法计数器:

74LS191和74LS193

同步集成计数器

- 74LS161/74LS160
- 同步4位二进制/同步十进制加法计数器

- $\overline{CR}$ —— 异步清零端
- $\overline{LD}$ —— 置数端（同步置数）
- $D_0 \sim D_3$ —— 数据输入
- $Q_0 \sim Q_3$ —— 数据输出
- CO —— 进位输出
- CT_P & CT_T —— 工作状态控制端

同步置数

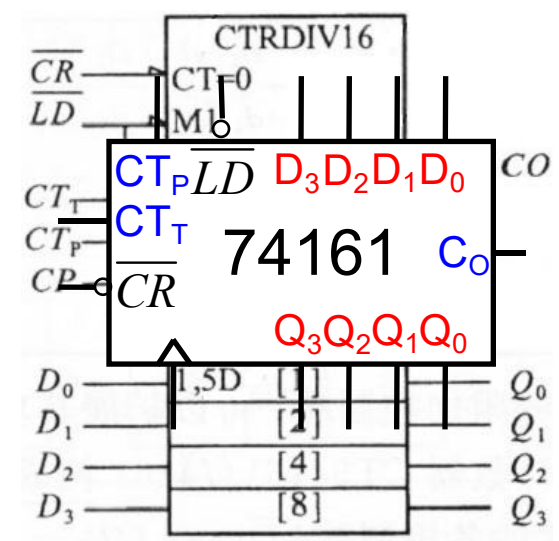
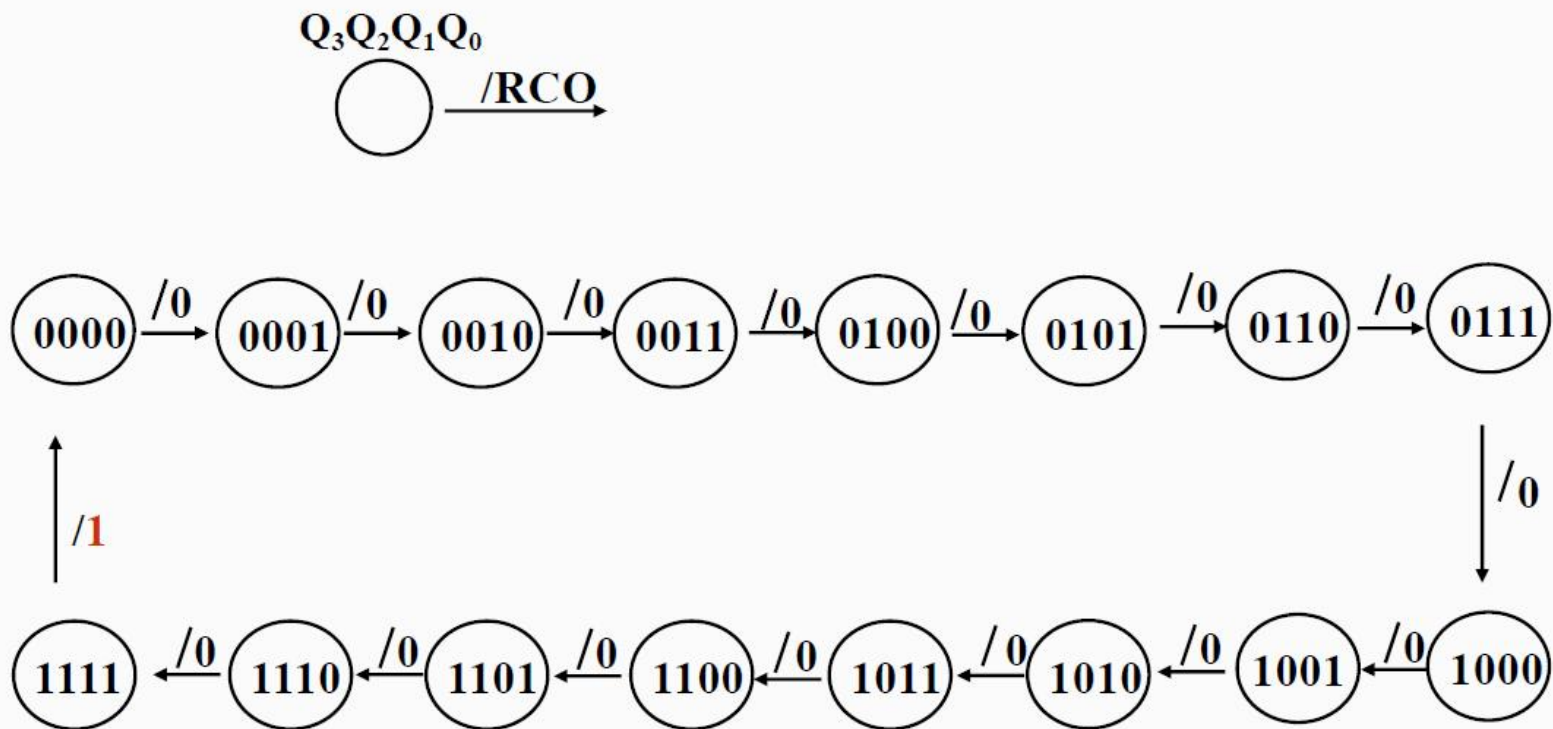


图6-2-24 CT54/74161 逻辑符号

表 6-2-9 CT54161/CT74161 (CT54160/CT74160) 功能表

输 入					输 出							
$\overline{CR}$	$\overline{LD}$	CT_T	CT_P	CP	D_0	D_1	D_2	D_3	Q_0	Q_1	Q_2	Q_3
0	×	×	×	×	×	×	×	×	0	0	0	0
1	0	×	×	↑	d_0	d_1	d_2	d_3	d_0	d_1	d_2	d_3
1	1	1	1	↑	×	×	×	×	计 数			
1	1	0	×	×	×	×	×	×	触发器保持, $CO=0$			
1	1	0	0	×	×	×	×	×	保 持			

同步集成计数器



74161状态转换图

同步集成计数器

设计举例

例 用74161组成十进制(N=10)计数器

解： 先将74161接成M16计数器，

$\overline{CR}, \overline{LD}, CT_T, CT_P$ 均=1

然后作跳过六个状态 ($M-N=16-10=6$) 的十进制计数器，将模M计数器变为模N计数器。

用 $\overline{CR} = 0$ 实现反馈置0。

Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
⋮			⋮
1	0	0	1
1	0	1	0
1	0	1	1
⋮			⋮
1	1	1	1

☆ 选择初态为0，0~9为有效状态，10~15为无效状态。

☆ 当输入十个CP脉冲， $Q_3Q_2Q_1Q_0=1010$ 时，用1010作反馈，使 $\overline{CR}=0$ ，强制计数器置0。强制置0信号是异步置0，与计数脉冲CP无关。因而1010这个状态不计算在主循环内。

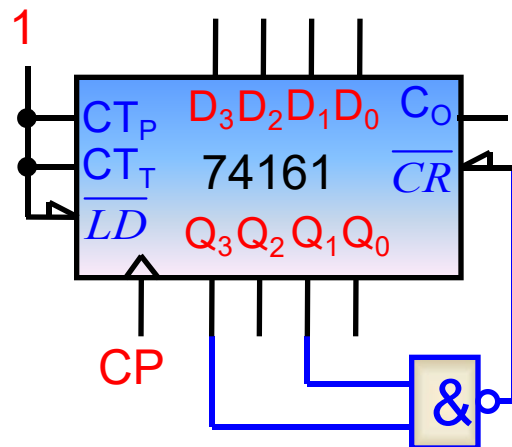
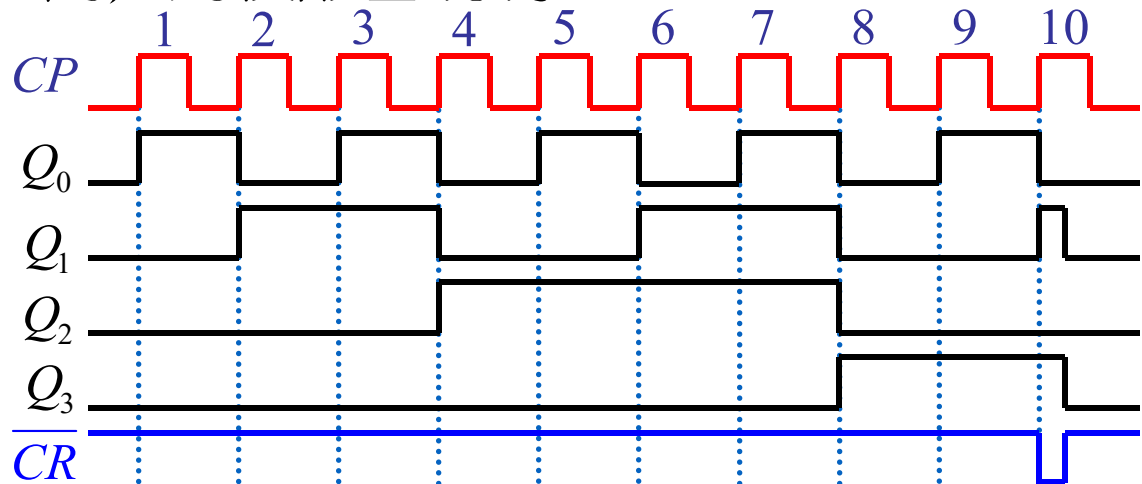
$$\overline{CR} = \overline{Q_3 Q_1} = 0$$

反馈电路是一个二输入与非门

同步集成计数器

最后画出原理电路图

☆ 为什么1010状态不算在主循环内，用波形图说明

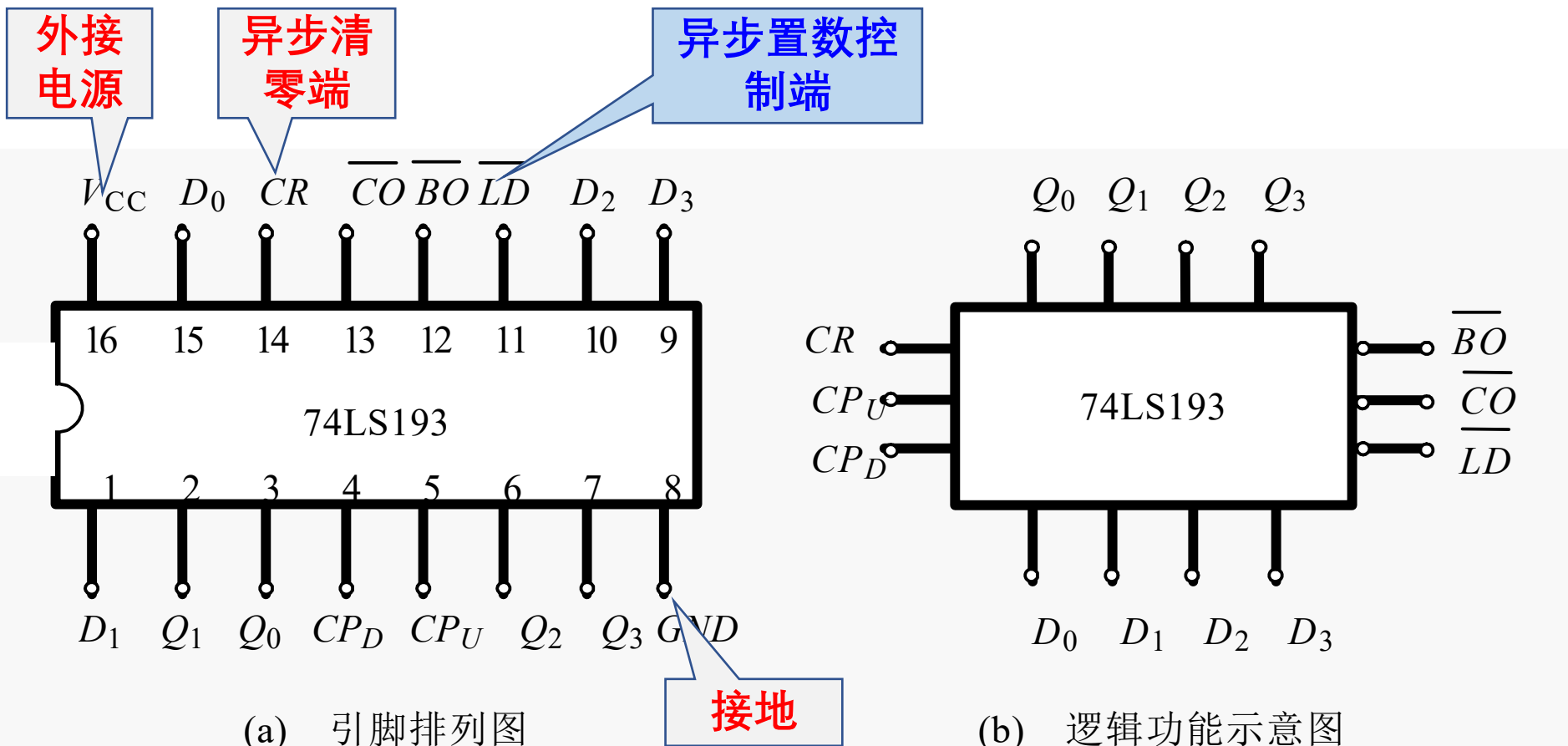


缺点： $\overline{CR}$ 脉冲宽度太窄，清0不可靠。

当第十个脉冲上升沿到达后 $Q_3Q_2Q_1Q_0 = 1010$, $\overline{CR} = 0$ 。只要 $\overline{CR} = 0$, 计数器强制置0。 $Q_3Q_2Q_1Q_0 = 1010$ 只维持了很短的时间，在第十个脉冲的大部分时间里， $Q_3Q_2Q_1Q_0 = 0000$ ，且当第十一个脉冲到达时，计数器又计到1，所以1010只能看作暂态，不算在主循环内。

在反馈电路清0信号的作用下，计数器产生0~9的循环，因此，形成模十的8421BCD码计数器

双时钟二进制同步可逆计数器74LS193



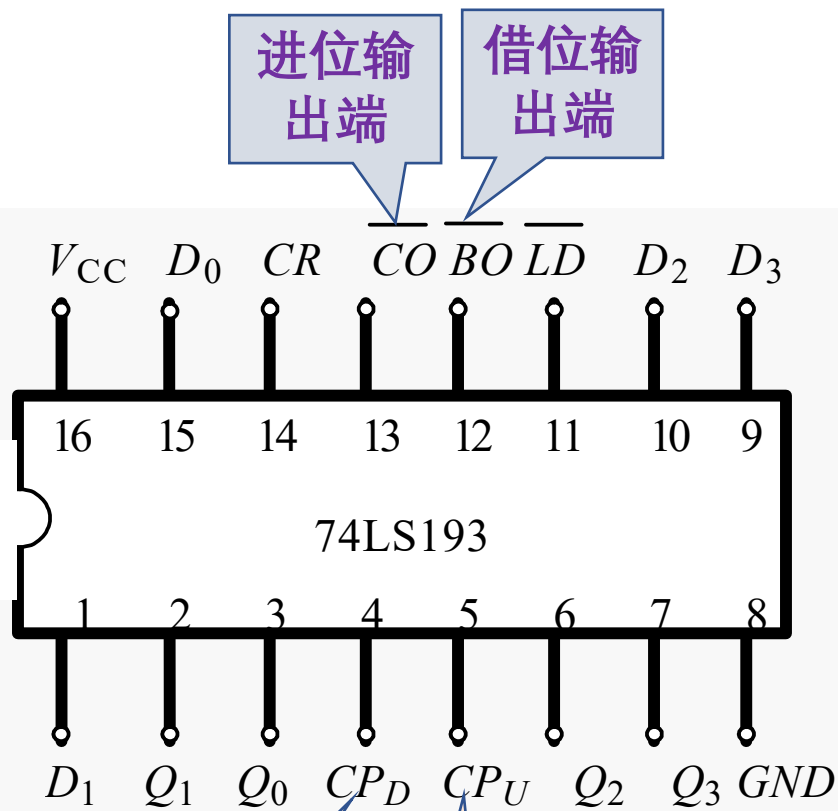
(a) 引脚排列图

(b) 逻辑功能示意图

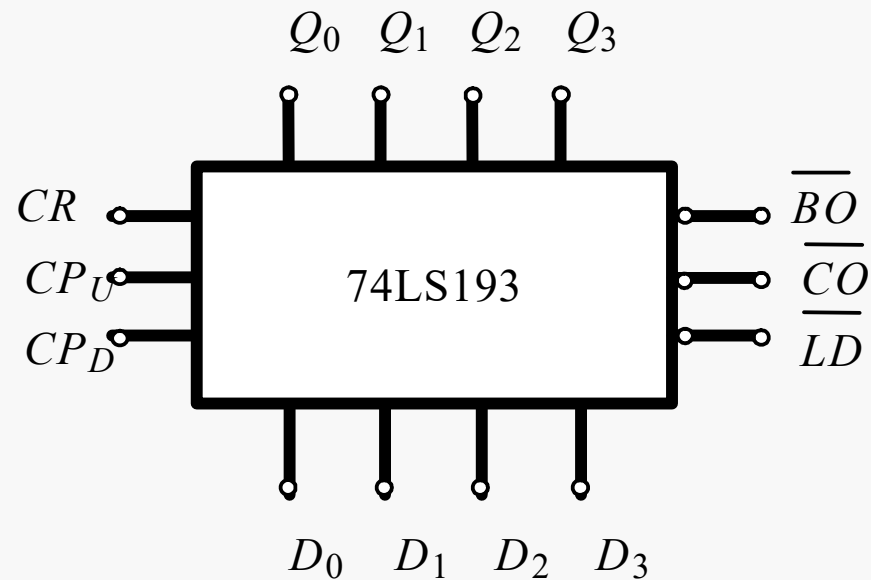
$D_3D_2D_1D_0$: 外部输入数据
 $Q_3Q_2Q_1Q_0$: 计数器输出数据

- 当 $CR=1$ 时, $Q_3Q_2Q_1Q_0=0000$
- 当 $CR=0$ 且 $\overline{LD}=0$ 时,
 $Q_3Q_2Q_1Q_0 = D_3D_2D_1D_0$

双时钟可逆十六进制计数器74LS193



(a) 引脚排列图



(b) 逻辑功能示意图

双时钟可逆十六进制计数器74LS193

74LS193的逻辑功能表

CR	$\overline{LD}$	CP_U	CP_D	D_3	D_2	D_1	D_0	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
1	×	×	×	×	×	×	×	0	0	0	0
0	0	×	×	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0
0	1	↑	0	×	×	×	×	加 1 计数			
0	1	0	↑	×	×	×	×	减 1 计数			
0	1	×	×	×	×	×	×	保持			

① **CR=1时，异步清零**

双时钟可逆十六进制计数器74LS193

74LS193的逻辑功能表

CR	$\overline{LD}$	CP_U	CP_D	D_3	D_2	D_1	D_0	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
1	×	×	×	×	×	×	×	0	0	0	0
0	0	×	×	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0
0	1	↑	0	×	×	×	×	加 1 计数			
0	1	0	↑	×	×	×	×	减 1 计数			
0	1	×	×	×	×	×	×	保持			

② $CR=0$ ， $\overline{LD}=0$ 时，异步置数

$$Q_3 \sim Q_0 = D_3 \sim D_0$$

双时钟可逆十六进制计数器74LS193

74LS193的逻辑功能表

CR	$\overline{LD}$	CP_U	CP_D	D_3	D_2	D_1	D_0	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
1	×	×	×	×	×	×	×	0	0	0	0
0	0	×	×	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0
0	1	↑	0	×	×	×	×	加 1 计数			
0	1	0	↑	×	×	×	×	减 1 计数			
0	1	×	×	×	×	×	×	保持			

③ $CR=0$ ， $\overline{LD}=1$ ， CP_U ↑ 且 $CP_D=0$ 时，加1计数

- 当加法计数到达1111时，进位输出 $\overline{CO}$ 输出一个脉宽等于 CP_U 的低电平部分的低电平脉冲；当下一个 CP_U 上升沿到来时， $\overline{CO}$ 变为高电平。由此产生一个 $\overline{CO}$ 的上升沿。
- 多个74LS193级联时，将低位的 $\overline{CO}$ 端与高位的 CP_U 端连接起来。

双时钟可逆十六进制计数器74LS193

74LS193的逻辑功能表

CR	$\overline{LD}$	CP_U	CP_D	D_3	D_2	D_1	D_0	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
1	×	×	×	×	×	×	×	0	0	0	0
0	0	×	×	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0
0	1	↑	0	×	×	×	×	加 1 计数			
0	1	0	↑	×	×	×	×	减 1 计数			
0	1	×	×	×	×	×	×	保持			

④ $CR=0$, $\overline{LD}=1$, $CP_D \uparrow$ 且 $CP_U=0$ 时, 减1计数

- 当减法计数值为0000时, 借位输出 $\overline{BO}$ 输出一个脉宽等于 CP_D 的低电平部分的低电平脉冲; CP_D 的下一个上升沿到达时, $\overline{BO}$ 变为高电平; 由此产生一个 $\overline{BO}$ 的上升沿
- 多个74LS193级联时, 将低位的 $\overline{BO}$ 端与高位的 CP_D 端连接起来

双时钟可逆十六进制计数器74LS193

74LS193的逻辑功能表

CR	$\overline{LD}$	CP_U	CP_D	D_3	D_2	D_1	D_0	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
1	×	×	×	×	×	×	×	0	0	0	0
0	0	×	×	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0
0	1	↑	0	×	×	×	×	加 1 计数			
0	1	0	↑	×	×	×	×	减 1 计数			
0	1	×	×	×	×	×	×	保持			

⑤ $CR=0$, $\overline{LD}=1$, CP_D 和 CP_U 均无变化时, 状态保持

第五章习题

17. 用 $J-K$ 触发器和门电路设计一个七进制加法计数器。
18. 用 Verilog HDL 编写一个模块，实现加减可控的九进制计数器。



武汉大学
Wuhan University

谢谢!

2025-11-20

